

chap 3: Internet Principles

Page No.:

Date:

IP (Internet Protocol):

- IP is a protocol used to send data from one machine (or device) to another over the internet. Data is packaged into units called packets, which contain both the data being sent and addressing information.
- Packet Structure: Each IP packet includes:
 - (1) Destination Address: The address of the machine that is intended to receive the data.
 - (2) Source Address: The address of the machine that is sending the data.
 - (3) Data: The actual content being transmitted.

Routing and Intermediary Machines:

- Routing: When data is sent, the sending machine does not always know the best route to the destination. Instead, packets often pass through multiple intermediary machines known as routers. Routers direct the packages along the most efficient path to reach their destination.
- Diverse Networks: The underlying networks can vary widely. Data packets can be transmitted over different types of networks, including:
 - (1) Wired network (ex. Ethernet)
 - (2) Wireless network (ex. Wi-Fi)
 - (3) Telephone systems
 - (4) Satellite links

Reliability and Limitations:

- Page No.:
Date:
- 1) No guarantees: One of the key characteristics of IP is that it does not guarantee delivery of packets. Just like if a message sent to a cousin might get stuck if they are unaware of the intended recipient, an IP packet may not reach its destination. There is no built-in mechanism to confirm whether the packet arrived successfully.
 - 2) Packet size limitations: Additionally, there are limitations on the amount of data that can fit into a single packet. If the data exceeds this limit, it must be split into multiple packets, which can complicate the transmission process.

TCP (Transmission Control Protocol):

- 1) TCP addresses: the need to send longer messages that exceeds the capacity of single IP packet (analogous to sending multiple postcards). It allows for the transmission of larger amounts of data by breaking it into manageable segments.
- 2) Reliability: TCP ensures that messages are delivered reliably. It provides mechanisms to confirm that data has been received correctly, which is essential for applications where data integrity is critical.

Key features of TCP:

- (1) Sequence Numbers: Each segment of data sent using TCP is assigned a sequence number, this allows the receiving end to reassemble the segments in the correct order, even if they arrive out of sequence. This is particularly useful when messages are sent over multiple days or through different routes.

- (2) Acknowledgements: When a segment is received, the receiving device sends back an acknowledgement (ACK) to the sender. This notification indicates which segments have been successfully received. If the sender doesn't receive an acknowledgement for a particular segment, it knows that the segment may have been lost in transit and can resend it.
- (3) Retransmissions: If a segment is determined to be missing (that is, no acknowledgement is received), TCP automatically retransmits that segment. This ensures that all parts of the message eventually reach the destination intact.

TCP and IP Combination:

- (1) Layered Protocols: TCP is built on top of the IP protocol, meaning that it relies on IP for the basic transmission of packets across networks. While IP handles the routing and delivery of packets, TCP adds the reliability and ordering features necessary for effective.
- (2) Applications Built on TCP: The combination of TCP and IP is foundational for many Internet services. For example,
- Email: Email protocols (like SMTP) use TCP to ensure that messages are sent and reliably received.
 - HTTP: The Hypertext Transfer Protocol (HTTP), which is used for transmitting web pages, also relies on TCP to ensure that data is delivered accurately and in the correct order.

The TCP/IP Protocol Suite (Diagram).

- The low-level protocols at the link layer manage the transfer of bits of information across a telephone network, or even by short-range radio standards such as IEEE 802.15.4 designed to carry data over the personal area network (PAN), that is to say between devices carried by an individual.
- The Internet layer then sits on top of these various links and abstracts away the gory details in favour of a simple destination address.
- Then TCP, which lives in the transport layer, sits on top of IP and extends it with more sophisticated control of the message passed.
- Finally, the application layer contains the protocols that deal with fetching web pages, sending emails, and internet telephony. Of these, HTTP is the most ubiquitous for the web, and indeed for communication between Internet of Things devices.

UDP (User Datagram Protocol):

- As you can see, TCP is not the only protocol in the transport layer.
- Unlike TCP, but as with IP itself, in UDP each message may or may not arrive. No handshake or retransmission occurs, nor is there any delay to wait for messages in sequences.
- These limitations make TCP far preferable for many of the tasks that IOT devices will be used for. The lack of overhead, however, makes UDP useful for applications such as streaming data, which can cope with minor errors but doesn't like delays.

- Voice Over IP (VoIP) - computer-based telephony, Such as Skype - is an example of this: missing one packet might cause a tiny glitch in the sound quality, but waiting for several packets to arrive in the right order could make the speech too jittery to be easy to understand.
- UDP is also the transport for some very important protocols which provides common, low-level functionality, such as DNS and DHCP, which relate to the discovery and resolution of devices on the network.

Difference between: TCP and UDP and IP

TCP	UDP	IP
1) Connection-Oriented protocols.	Datagram oriented protocols.	
2) The data delivery is guaranteed and hence it's reliable.	The data delivery isn't guaranteed & hence not completely reliable.	
3) Data Sequencing takes place.	No data sequencing.	
4) It is comparatively slower.	RTSP (Real-Time Streaming Protocol).	
5) The header length is 20-80 bytes.	The fixed length header is 8 bytes.	
6) Offers congestion control solution.	Doesn't offer congestion control sol ⁿ .	
7) Error checking and error recovery takes place.	Basic error checking is done, but no error recovery processes takes place.	

8) Heavier compared to UDP and requires 3 data packets before. Setting up a connection for data transmission.	If it's a light-weight.
---	-------------------------

DNS (Domain Name System):

- While computers operate using numerical, IP addresses (32-bit numbers), these addresses are difficult for humans to remember. DNS provides a way to use familiar domain names (like google.com) or (bbc.co.uk) instead of numerical addresses, making it easier for users to access websites and services.

Structure of Domain Names:-

- Domain Names: A domain name consists of several parts, including:

(a) Top-Level Domain (TLD): The last part of the domain names, such as, .com, .uk, or .cc. TLDs categorize domains and help organize the internet.

(b) Subdomains: Domains can be further divided into subdomains. For example, bbc.co.uk has bbc as the second-level domain and .co.uk as the TLD.

(c) www.google.com

└── Top-Level Domain (TLD): .com
 └── Second-Level Domain (SLD): .google
 └── Subdomain: www

(d) Hierarchical Structure: The DNS Operates in a hierarchical manner. Each TLD knows where to find information about the domains within it. For example, the .com TLD knows how to locate google.com, and Wiley.com.

Difference between Static IP and dynamic IP Address:

Static IP Address		Dynamic IP Address	
(1)	It is provided by ISP (Internet Services provider).	(1)	While it is provided by DHCP (Dynamic host configuration Protocol).
(2)	Static IP Address doesn't change any time, it means if a Static IP address is provided then it can't be changed or modified.	(2)	While dynamic IP Address change any time.
(3)	Static IP Address is less Secure.	(3)	While in dynamic IP Address, there is low amount of risk than Static IP address risk.
(4)	Static IP Address is a difficult to designate.	(4)	While dynamic IP address is easy to designate.
(5)	The device designed by Static IP Address can be traced.	(5)	But the device designed by dynamic IP Address can't be traced.
(6)	Static IP Address is more stable than dynamic IP Address.	(6)	While dynamic IP Address is less stable than Static IP Address.

(7) The cost to maintain the Static IP Address is higher than dynamic IP Address.

(8) It is used where computational data is less confidential.

(9) Simplifies the troubleshooting as the IP is always the same.

(7) While the maintaining cost of dynamic IP Address is less than Static IP address.

(8) While it is used where data is more confidential and needs more security.

(9) While dynamic IP increases the complexity of diagnosing the network issues.

CHAPTER 1: Thinking About Prototyping

Prototyping:

- A prototype is a Preliminary Version of an IOT devices or System, built to test its functionality, design, and user experience before fullscale production.
- Why Prototyping is Essential?
 - Early problem Discovery: A prototype helps identify design issues before mass production. Modifying a single devices is easier and cheaper than recalling thousands.
 - Three parallel components to Build:
 - (1) The physical Thing - the object or devices itself.
 - (2) The Electronics - Sensors, Microcontrollers, etc, that make it "Smart".
 - (3) The Internet Service - The cloud or web platform, it connects to.
- What a prototype typically looks like?
 - Built Using prototyping Microcontrollers (ex., Arduino, Raspberry Pi).
 - Components wired together on a breadboard.
 - Housed in improvised cases (like tins or laser-cut boxes).
 - Not cost-effective or polished for commercial sale - but functional and demonstratable.
- Why prototype is valuable?
 - Test Usability and feasibility.
 - Showcase the product to friends, Stakeholders or investors.
 - Gain feedback before Scaling to manufacturing.

- Platform choice isn't One-Size-fits-All.
- There's no Universal Solution like "Always Use Arduino". The choice depends On:-
 - Project goals.
 - Technical needs.
 - Current Market trends.
 - Evolving Prototyping technology.

Sketching:

- In the context of IOT product development, Sketching goes beyond drawing ideas on paper - it's about rapidly exploring and testing concepts in hardware and software to understand what works and what's doesn't, this is especially valuable during the prototyping stage, where iteration is fast and fidelity (that is, how possible it is) matter less than functionality and learning.
- Sketching in this context includes:-

- Drawing early designs on paper to Visualize ideas.
- Using basic hardware materials like LEGO, Card-board, or foam for physical structure.
- Writing quick software Scripts or Servers Setups with minimal features.
- Creating rough electronics setups with Arduino or Raspberry Pi boards.

Real-World Example: The Printernet fridge

- This was a hackathon Prototype built during BERG's little printer event,
 - Goal $\Rightarrow$ Create an input device that adds items to a Semi-automated Shopping list, Which then prints On

a little printer.

• Design Breakdown $\Rightarrow$

- Graphic layout for printer content.
- Physical hardware to select items (push buttons).
- Server software to process inputs and publish content.
- Instead of building a polished system, the developer:
 - Used Sinatra, a light-weight Ruby web-framework, for quick backend API creation.
 - Built hardware with an Arduino Ethernet board, simple pushbuttons and a busy LED to manage network traffic visibility.
 - Made rapid iterations by showing the devices to others at the hackathon for feedback.
 - Used sticky notes as a placeholders UI to hint at future improvements (ex., barcode scanner supports).

Key Principles:

- Iterative design $\Rightarrow$ Start small, test, adopt.
- Low fidelity prototyping $\Rightarrow$ focus on functionality over polish.
- Rapid Integration $\Rightarrow$ Connecting hardware to server code with quick APIs.
- User feedback loop $\Rightarrow$ Informal testing to guide improvements.
- Scaffold approach $\Rightarrow$ Build minimal, modular components to prove a concept.

familiarity, cost and ease in IOT Prototyping:

(1) familiarity:

- Choose tools and languages, you already know.
- Example: If you know python, using a Raspberry Pi may be more efficient than learning Arduino/C++.
- Similarly, use frameworks or materials (eg foamcore, cardboard) you're already skilled with to speed up developing.

(2) Cost Vs Ease of Prototyping:

- There's trade-off between cost and ease.
- Cheapest Options (ex. raw AVR Microcontrollers) are difficult to use without technical skill.
- Platforms like Arduino (~£20) offer a balance $\Rightarrow$ affordable, easy to program and widely supported.
- Beagle Bone or Raspberry Pi (~£30) adds more power and easier coding with Linux support.
- Smartphones or PCs $\Rightarrow$ Expensive, but if already available, they can be used effectively for initial tests.

(3) Performance Vs. Prototyping:

- While PCs and Smartphones offer convenience, they may not handle hardware-level control (eg., Motors, LEDs) well.
- Microcontroller like Arduino offer direct access to the GPIO pins but requires understanding of hardware integration.
- Combining platforms $\Rightarrow$ Use Arduino for hardware control and a PC or phone for computation / Internet Connectivity.

(4) Developer Skill Matters:

- Arduino is a good for beginning in hardware.
- If you're comfortable with general programming (ex. python, JavaScript), using a general-purpose computer might

feels easier - but setup complexity increases.

Prototypes and Production in IOT Development :-

(1) Changing Embedded platforms:

- Prototypes, often use User-friendly, powershell platforms (like Arduino, Raspberry Pi, or even a PC or Smart-phones).
- But for mass production, You may need to Switch to:
 - Smaller, cheaper, more power-efficient Micro-controllers (Ex., bare AVR chips instead of full Arduino boards).
 - Custom PCBs instead of breadboards or development boards.
- Challenges during porting:
 - Limited Memory and processing (Ex, Arduino has only 2Kb and RAM).
 - Compatibility of I/O pins and peripherals.
 - Rewriting or Optimizing code to suit new constraints.
 - Learning new platforms, tools, or programming languages.

(2) Physical Prototypes and Mass Personalization:

- Prototype bodies are often made with 3D printing, cardboard, or form foam flexibility and Speed.
- For production, techniques like injection molding are used for cost effectiveness and durability.
- But With digital fabrication, mass personalization becomes possible:
 - Each device can be slightly different (Ex., name color, or function).

- Appears to customers wanting Unique productions.
- Justifies charging a premium price.
- However, logistics and cost per unit increases due to one-off custom parts.

(3) Scaling Server Software (Climbing into the cloud):-

- Compared to hardware, Software is easier to Scale:
 - The logic and architecture from a prototype can often be retained.
 - You may Upgrade framework (Ex., from Sinatra to Rails or Django) to support user accounts, Security and performance.
- Cloud platforms (Ex. AWS, Google cloud):
 - Allow dynamic scaling - resources expands Shrink - based on user demands.
 - Minimize the need for large upfront Server investments.

Open Source Vs. Closed Source in IOT projects :

- Open Source and closed Source approaches isn't just philosophical - It's deeply practical, especially when creating Internet of Things (IOT) devices that blend hardware and software. The choice affects your project's protection, growth, community ~~and~~ supports and even commercial visibility.
- Closed Sources: In a closed Source model, the creator keeps the Source code, Schematics, and designs private.

- Advantages:

- Protects commercial interest: Others can't easily replicate your work and complete directly.
- Asserts IP Ownership: Easier to enforce copyright trademarks, and patents.
- Default for big companies: Legal departments are already structured around IP protection.
- Control: You dictate how the product is used, adapted or redistributed.

- Disadvantages: (Drawbacks)

- Slower Collaboration and improvement cycle.
- No community contribute or bug fixing.
- Not as transparent, which can limit user trust - especially in IOT where data handling is critical.

Open Sources:

- What does it means to mix Open and closed Source?

- In an Open-source model, the creator shares the source code, hardware designs, or product space, publicly. Example on GitHub, Fritzting or thingiverse.

Benefits:

- Community growth: Other contribute ideas, fixes, features, etc.
- Reputation building: Showcases your skills and attract opportunities.
- Faster innovation: Open feedback loops from real user.

- Page No.:
Date:
- Ecosystem Effects: You get access to other's Open contributions in returns.

Drawbacks:

- Risks of other copying or commercializing your ideas.
- Many may need more time to clean and document the code/designs before sharing.
- Users may expect supports or maintenance, an even if it's free.

Open Source as a competitive advantages:

(1) Leverage community tools:

- You don't need to reinvent everything. Instead of:
 - Writing your own HTTPs stack.
 - Or building your own servo motor controller.
- You can use existing Open tools like Arduino libraries or Ruby or Rails, which are:
 - Tested by many.
 - Well-documented.
 - Supported by large communities.

(2) Build Mindshare:

- Products like Arduino become popular the best technically, but because,
 - They were Open and extendable.
 - They invited third-party products (ex. Shields).
 - They became platforms supported by a vibrant Ecosystems.

Open Sources as a Strategic Weapon:

• Commoditizing your complements:

- If your product depends on other products, you have might there, make those other products Open Source to drive their price down, which increases demands per for your own products.
- Ex., if you sell microcontrollers, Open Sourcing tools that make them easier to use (like programming libraries) helps sell more chips.
- Google Open-Sourced Android to Weapon Apple's IOS.
- Facebook Open-Sourced its data center design (Open computer) to Undermine google's edge in an Infra-structures.

Open Source: Roles and benefits

Notes

- | | |
|---|---|
| 1) Build developer trust & community: | Only is done transparently & With proper documentation. |
| 2) Reduces dev time via code reuse: | Use mature libraries and frameworks already tested by the community. |
| 3) Helps create platform ecosystems: | Arduino succeeded not by being best - but by being most adopt-ables. |
| 4) Offers Strategic business leverage: | Open-source what helps you and hurts competitors - but protected your core. |
| 5) Requires effort to maintain & support: | "Gift economy" means users will expect updates and engagements. |

Mixing Open and closed Source:

- You can release some components of your project Under an Open-Source license (ex. firmcoare, libraries).
- At the Same time, you can keep other parts closed, an especially those that:
 - contain core business logic.
 - Invoke commercial advantages.
 - Are not ready for public releases.
- This balance always allows, you to benefits from the Open-Source community where still protecting your IP where necessary.

Why choose a Hybrid Model?

- Advantages of Open-components:
 - Gain community trust and feedback.
 - Encourage community contributions (bug fixes, features).
 - Improve Visibility and adoption.
- Why keep some parts closed?
 - protect competitive advantage or Unique IP.
 - Avoid extra work polishing messy code for public releases.
 - Meet client or business requirements.

Closed Source for Mass Market Products:

When Why keep it closed?

- If your product has mass market potential, like a Smart thermostat or energy meter, closed source becomes more

attractive for several reasons:

- Speed to Market: The first one to market at scale often wins. Keeping the tech-closed helps protect your lead.
- Supply Chain Advantages: In mass product, control over manufacturing, logistics, and pricing is as important as the product itself.
- Production from copycats: Open sourcing everything makes it easy for competitors - especially well-equipped ones - to clone your product without the cost of R & D or reverse engineering.
- Example: Nest (Smart thermostat): If an engaged energy company rolls out millions of units, having the code and schematics public could make it easy for competitors to replicate and undercut the product.

Tapping into the Community:

- Why Community Still Matters (Even for closed Source):
 - Even if your core is closed, building with or on a popular open platform (like Arduino or Raspberry Pi) gives you access to:
 - forums, tutorials, blog posts: helpful resources when you're stuck.
 - In-person support at maker meetups or hackerspaces.
 - Mentorship from experienced makers makers or hobbyists.
 - Searchable knowledge: You can Google "arduino servo potentiometer" and get instant help.